CS 230 Project — Draw It or Lose It

Software name: Draw It or Lose It — Web Edition

Version: 1.0

Date: November 21, 2025

Author: Syneca Kindle

Document Revision History

Version	Date	Author	Comments
1.0	11/21/2025	Syneca Kindle	Initial filled software design document for Project One submission.

Executive Summary

Draw It or Lose It (Web Edition) is a multi-platform reimplementation of the client's existing Android-only game. The goal is to deliver a web-based, real-time multiplayer game that supports multiple teams and players, enforces unique game/team names, and follows the single-instance-in-memory constraint for the game service. This document outlines the design constraints, domain model, and recommended platform choices, and provides an initial plan for implementing the Java backend classes (Entity, Game, Team, Player) and a singleton GameService using iterator and singleton patterns. The design emphasizes real-time communication (WebSocket), scalability via containerization, and secure user data handling.

Critical next steps for the client: confirm target concurrency (expected simultaneous players), preferred cloud provider (if any), and whether the original drawing asset library is to be reused or replaced.

Requirements

Business requirements

- Provide a web-based version of Draw It or Lose It that can serve multiple platforms (desktop browsers, mobile browsers).
- Support games with one or more teams; each team may have multiple players.
- Ensure game/team names are unique so users can check name availability.

- Game sessions should play as four rounds, each one minute long, with rendering finishing at 30 seconds and a 15-second final-guess window.

Technical requirements

- Only one instance of the GameService class should exist in memory (singleton).
- Entities (Game, Team, Player) must have unique identifiers and names.
- Use patterns: singleton for GameService, iterator for traversing collections (used by add/get operations).
- Java-based backend (starter code provided). Must implement Entity base class and refactor Game, Team, Player to inherit from it.
- Provide a ProgramDriver main method for testing and demonstration.

Design Constraints

1. Single JVM-instance singleton constraint — only one GameService instance must exist in JVM memory. Implication: implement thread-safe singleton (e.g., synchronized lazy initialization or enum singleton) and consider serialization/cluster-awareness for scaling (singleton is per JVM; for multi-node deployments, additional coordination is required).

2. Unique names requirement — Game and Team names must be globally unique within the running instance. Implication: methods that add games/teams must check existing names (use iterator pattern or maintain a name-index such as a HashSet for O(1) checks).

3. Real-time gameplay — real-time drawing/clue rendering and guess flow require low-latency communication; use WebSockets or similar for server-client push updates.

4. Cross-platform client compatibility — clients will access via browsers on desktop and mobile, so UI must be responsive; server must present REST APIs + WebSocket endpoint.

5. Asset management — large drawing library implies storage/bandwidth constraints; use cached CDN/object-store for static assets.

6. Concurrency & state — games are stateful (round timers, rendered frames). Implication: server must manage per-game state safely (thread-safety, atomic updates, and proper cleanup when games end).

7. Security & privacy — must protect player identities and connection tokens; secure transport (TLS) and proper session/auth workflows are required.

8. Persistence vs in-memory — initial spec implies active in-memory game instance; decide what to persist (e.g., historical games, user accounts) vs ephemeral game state.

System Architecture View

- Client tier: browser-based UI (responsiveness for desktop & mobile). Communicates over HTTPS + WebSocket.
- Application tier: Java backend (servlets or Spring Boot) hosting REST endpoints and WebSocket handlers. Contains singleton GameService managing in-memory games, teams, players.
- Persistence tier: relational DB (PostgreSQL) for persistent objects (user accounts, completed games history), object store (S3-compatible) or CDN for static drawing assets.
- Infrastructure: containerized services (Docker) orchestrated by Kubernetes or cloud-managed containers for scalability; load balancer and TLS termination; optional Redis for session/locking if running multiple app instances.

Domain Model

Overview of UML classes (as provided in starter material):

- Entity (base class)
  - Attributes: id (unique identifier), name
  - Purpose: common properties for Game, Team, Player. Promotes reuse and enforces consistent naming/id semantics.

- Game (extends Entity)
  - Attributes: list of Team, game state (round number, timer, active puzzle), unique game id.
  - Behavior: startRound(), endRound(), renderDrawing(), acceptGuesses().

- Team (extends Entity)
  - Attributes: list of Player, team score.
  - Behavior: addPlayer(), removePlayer(), submitGuess().

- Player (extends Entity)
  - Attributes: connection/session id, role (e.g., guesser), possibly persistent user id if accounts used.
  - Behavior: sendGuess(), receiveUpdates().

- GameService (singleton manager)
  - Holds collections of Game instances; static counters for next id values. Exposes addGame(), getGame(), addTeam(), addPlayer(), etc. Must be implemented as a thread-safe singleton and use iterator pattern for searches and additions.

OOP principles demonstrated

- Inheritance: Entity base reduces duplicated code for id/name across Game/Team/Player.
- Encapsulation: each class exposes behaviors and hides internal state (use private fields + getters/setters).
- Polymorphism: common Entity type allows collections and methods to treat

Game/Team/Player uniformly where appropriate.
- Single Responsibility: classes focus on single roles (Game handles timing & game logic; Team manages players and scoring).
- Design Patterns: Singleton (GameService), Iterator (used to traverse collections to find uniqueness), Factory (optional: for creating Entities), and possibly Observer (for notifying connected clients of game state changes).

Evaluation

Server Side

Mac: macOS is a solid development environment with good tooling (Xcode, Homebrew) and native Docker support. For production, macOS servers are uncommon and expensive; choose macOS only for local dev/test.

Linux: Strongest server-side candidate — mature, performant, good container orchestration support, low cost and widely used in production. Best compatibility for Java runtime, Docker, Kubernetes.

Windows: Works for Java and .NET stacks; Windows Server is viable but less common for container/K8s production; slightly higher footprint and licensing cost.

Mobile Devices: Mobile devices are clients, not servers. They cannot (practically) host the server components; however edge/local sessions could be supported for P2P but unnecessary for this app.

Client Side

Mac: Browser testing on macOS (Safari + Chrome) should be included; development tools readily available.

Linux: Good for testing cross-browser behavior; fewer proprietary browsers but Chrome/Firefox available.

Windows: Must test extensively on Windows browsers (Edge, Chrome, Firefox) since many users will run Windows.

Mobile Devices: Must ensure responsive UI for Android and iOS browsers; consider progressive web app (PWA) to enhance mobile UX and allow offline-like behaviors.

Development Tools

Mac: IDEs: IntelliJ IDEA, Eclipse; Node.js/npm for frontend; Docker Desktop; Postman.
Linux: IntelliJ/Eclipse, maven/gradle, Docker, kubectl; command line tools ideal.
Windows: IntelliJ/Eclipse, Docker Desktop (WSL2 recommended), Node.js, browsers for testing.

Mobile Devices: For native testing: Android Studio (Android); Xcode (iOS) for native wrappers. For web clients: Chrome DevTools and remote device emulation, cross-browser testing tools.

Recommendations

Operating Platform

Primary recommendation: Host server components on Linux-based cloud instances or containers (Ubuntu or Amazon Linux). Linux gives the best combination of cost, tooling, community support, and production compatibility.

Operating Systems Architectures

- Use 64-bit Linux servers with container orchestration (Docker images, optionally Kubernetes). Host Java app (Spring Boot or plain Java) in containers behind a load balancer. For small-scale use a single container with horizontal scaling via Kubernetes or ECS.

Storage Management

- Persistent metadata: PostgreSQL for relational data (users, teams, saved game history).
- Static assets: object store (AWS S3 or S3-compatible provider) + CDN for drawing image library to reduce latency.
- Cache: Redis for session caching and ephemeral game state acceleration if needed.

Memory Management

- Use JVM best practices (heap limits tuned per container). The server should avoid storing excessively large drawing frames in memory: stream assets, and keep only light-weight metadata in-memory. Configure GC (G1 or ZGC where available) for low pause times; set container memory limits and appropriate JVM flags.

Distributed Systems and Networks

- Real-time communication: WebSockets (or socket.io) to deliver rendered drawing frames and guess notifications. Fall back to long-polling if necessary.
- API: RESTful API for resource management (create game, list teams).
- Scaling: For multi-node deployments, share session/locking state using Redis (pub/sub) or an external coordination service so the singleton-logic per JVM is reconciled across nodes (i.e., the singleton is per-JVM; coordinate multi-instance deployments via centralized store). Consider sticky sessions for WebSocket affinity or use a centralized message broker.
- Resilience: Implement reconnection logic on clients, graceful degradation if connection lost, and state persistence for active games when node crashes.

Security

- Transport: TLS (HTTPS/WSS) mandatory.
- Authentication/Authorization: Use short-lived tokens for game sessions (JWT or server-

issued tokens). Limit privileges appropriately.
- Input validation & rate limiting: Validate guesses and user input server-side; protect endpoints from abuse.
- Data protection: Hash/secure any credentials, minimize PII stored, enforce least privilege in database access.
- Monitoring & logging: Centralized logs and alerting for suspicious activity.

Implementation Notes & Best Practices (Java-specific)

- Entity class: implement Entity as an abstract class with protected final long id; protected String name; and proper constructors/getters/setters. Provide equals() and hashCode() based on id.
- ID generation: Keep static counters in GameService that produce unique ids. For production, prefer UUIDs or DB-generated ids for persistence.
- Singleton GameService: implement thread-safe lazy singleton (e.g., private static volatile GameService instance; public static GameService getInstance() { ... }). Synchronize write operations or use concurrent collections.
- Iterator pattern: use iterator or for-each loops over the internal collection when implementing addGame(), getGame() so name checks are explicit; alternatively, maintain supplemental HashMap<String, Game> to speed name lookups. Document whichever approach you use and keep it consistent.
- Thread-safety: use ConcurrentHashMap and CopyOnWriteArrayList or synchronize around mutation methods for game/team/player collections.
- Testing: use JUnit for unit tests. Use ProgramDriver.main() to perform manual flow tests (create game, add teams/players, simulate rounds).

Evaluation Summary (short)

- Best production platform: Linux containers in cloud (cost-effective, scalable).
- Frontend: browser-based responsive UI (React/Angular/Vue) using WebSockets for real-time updates; implement PWA for mobile-like UX.
- Storage: PostgreSQL + S3 + Redis for caching/coordination.
- Security: TLS, token auth, server-side validation, rate limiting.